

THE PAYMENT SCHEDULING PROBLEM[†]

Richard C. Grinold

*Center for Research in Management Science
University of California
Berkeley*

ABSTRACT

Large complicated projects with interdependent activities can be described by project networks. Arcs represent activities, nodes represent events, and the network's structure defines the relation between activities and events. A schedule associates an occurrence time with each event; the project can be scheduled in several different ways. We assume that a known amount of cash changes hands at each event. Given any schedule the present value of all cash transactions can be calculated. The payment scheduling problem looks for a schedule that maximizes the present value of all transactions.

This problem was first introduced by Russell [2]; it is a nonlinear program with linear constraints and a nonconcave objective. This paper demonstrates that the payment scheduling problem can be transformed into an equivalent linear program. The linear program has the structure of a weighted distribution problem, and an efficient procedure is presented for its solution. The algorithm requires the solution of triangular systems of equations with all matrix coefficients equal to ± 1 or 0.

INTRODUCTION

This paper considers a financial scheduling problem introduced by Russell [2] in his paper, "Cash Flows in Networks." Given a project graph (of CPM or Pert type), the problem is one of scheduling events to maximize the present value of the outlays or receipts that occur at each event. Russell proposed a solution procedure and presented interesting interpretations of the problem, duality relations, and the solution procedure itself.

The principle results of this paper show that the scheduling problem (a nonlinear, nonconcave program) is equivalent to a linear program. This fact is then used to demonstrate that the optimal solution of the scheduling problem corresponds to a tree in the project graph, i.e., an extreme point in the set of feasible schedules. It follows that every local maximum is indeed a global maximum; thus the Kuhn-Tucker conditions are necessary and sufficient for optimality. This in-depth examination of the problem's special structure leads to an efficient solution procedure. The algorithm is related to Markowitz's special procedure for the weighted distribution problem [1, pp. 424-431]. We introduce a project deadline parameter into Russell's model. Our first algorithm solves the problem for any given project deadline. A second procedure finds the optimal solution for all possible deadlines. This yields a curve explicitly showing the tradeoff in project duration and present value.

There are four sections and an appendix. The appendix contains an example which illustrates most of the points in the paper. The first two sections discuss the problem and the equivalent linear program. The last two sections are given over to solution procedures.

[†] This work was supported, in part, by the National Science Foundation, Grant GS-3143.

THE PROBLEM

This section describes the payment scheduling problem while introducing notation and definitions. The concepts are illustrated by an example in the appendix. Readers who are unfamiliar with project graphs should consult Weist and Levy's excellent book [3].

$(V, \mathcal{A})$ is a directed graph. The nodes, $V = \{1, 2, \dots, N\}$, correspond to events, and the directed arcs, $\mathcal{A}$, correspond, with one exception, to activities.

Events 1 and N are special. Event 1 is the initiation of the project, and event N is the completion of the project. Throughout, collections of nodes will be designated by capital Roman letters, collections of arcs by capital script letters.

A real number, D_{ij} , is associated with each activity $(i, j) \in \mathcal{A}$; D_{ij} is the duration of activity (i, j) . With one exception the D_{ij} are nonnegative. The special arc, $N1$, links the completion event N and initiation event 1. D_{N1} is a negative number: $-D_{N1} = \delta$ is an upper bound on the duration of the entire project.

Let D represent the vector with elements D_{ij} for $(i, j) \in \mathcal{A}$. The triple $(V, \mathcal{A}, D)$ forms a *project graph* if: (i) each event i is on a directed path from event 1 to event N ; (ii) each directed cycle contains arc $(N, 1)$ and, the sum of the durations around any directed cycle is nonpositive.

It is useful to study the set of events immediately preceding and following each event. Let P_i be the set of events immediately preceding event i and F_i the events immediately following event i ; j is in P_i if and only if $(j, i) \in \mathcal{A}$; j is in F_i if and only if $(i, j) \in \mathcal{A}$. When $(V, \mathcal{A}, D)$ is a project graph, the sets P_i and F_i are nonempty, $P_1 = \{N\}$, $F_N = \{1\}$, and we can assume that $j \in P_i (i \neq 1)$ implies $j < i$.

Variables T_i for $i = 1, 2, \dots, N$ are associated with each event. T_i is called the epoch or occurrence time of event i . For $i \neq N$, T_i is the instant at which activities (i, j) for $j \in F_i$ are initiated. Activity (i, j) is then completed at epoch $T_i + D_{ij}$. We impose the condition that activities starting at event j cannot begin until all activities initiated at events $i \in P_j$ have been completed. Algebraically, this means

$$T_j - T_i \geq D_{ij} \quad \text{for } j = 2, 3, \dots, N \\ i \in P_j.$$

The duration of the project is $T_N - T_1$. The duration is limited by

$$T_1 - T_N \geq D_{N1}.$$

We always assume $T_1 = 0$, and call the $N - 1$ vector $T = (T_2, T_3, \dots, T_N)$ a *schedule*. Note the last constraint becomes $T_N \leq \delta$.

Let E be the node arc incidence matrix for $(V, \mathcal{A})$ with the (redundant) row corresponding to event 1 deleted. The schedule constraints are

$$TE \geq D.$$

If the D_{ij} are viewed as distances, the length of the longest path from event 1 to event N gives the briefest possible duration of the project. Let δ_0 be that duration. The fact that $(V, \mathcal{A}, D)$ is a project graph indicates that $\delta_0 \leq \delta$. The early start time ([3], pg. 27) for event i is the earliest possible epoch at which event i could occur; it is equal to the length of the longest path from node 1 to node i . The late

start time ([3], page 29) is the latest possible epoch of event i that is consistent with completion of the project in minimum time ($T_N = \delta_0$).

At epoch T_i a sum of money q_i changes hands; q_i is positive for receipts, negative for payments. The value of money is discounted at rate α . Thus the present value of the transaction at epoch i is $q_i \exp(-\alpha T_i)$. We assume $q_1 = 0$. The present value of any schedule T is given by

$$f(T) = \sum_{i=1}^N q_i \exp(-\alpha T_i).$$

The payment scheduling problem asks us to

$$(I) \quad \text{Maximize } f(T)$$

$$\text{Subject to } TE \geq D.$$

Notice three things about problem (I): (i) it has a feasible solution if and only if $\delta_0 \leq \delta$; (ii) if $\delta = +\infty$ (no time limit on the project), then the maximum may not be achieved; e.g., if $q_N = -1$; and (iii) when the q_i differ in sign the objective function can fail to be concave (or pseudoconcave); local maxima might exist.

THE TRANSFORMED PROBLEM

In this section, it is demonstrated that problem (I) is equivalent to a linear program.

The equivalent problem is

$$(II) \quad \text{Maximize } \sum_{i=1}^N y_i q_i$$

$$\text{Subject to } y_j K_{1j} \leq 1 \quad \text{for } j \in F_1$$

$$y_j K_{ij} - y_i \leq 0 \quad \text{for } i = 2, \dots, N; j \in F_i$$

$$-y_N \leq -K_{N1}.$$

The constants K_{ij} , $(i, j) \in \mathcal{A}$, are defined by $K_{ij} = \exp(\alpha D_{ij})$; thus $K_{ij} \geq 1$ if $(i, j) \neq (N, 1)$ and $0 < K_{N1} < 1$. The constraints of (I) are $T_j - T_i \geq D_{ij}$ for $(i, j) \in \mathcal{A}$, where $T_1 = 0$. The constraints of (II) are then $y_j K_{ij} - y_i \leq 0$ for $(i, j) \in \mathcal{A}$, where y_1 is equal to 1.

Let $y = (y_2, y_3, \dots, y_N)$ be an $N-1$ vector. We define a one-to-one correspondence between the nonnegative vectors $T \geq 0$ and the positive vectors y , where $y_i \leq 1$ for all i :

$$y_i = \exp(-\alpha T_i)$$

or

$$T_i = \ln(y_i) / -\alpha$$

Let $y(T)$ be the y determined by T .

THEOREM 1: (*Equivalence of (I) and (II)*).

(a) *T is a feasible solution of (I) if and only if $y(T)$ is a feasible solution of (II).*

(b) *T is optimal if and only if $y(T)$ is optimal.*

PROOF: Note that (b) follows easily from (a) since $f(T) = y(T)q$. Let $y = y(T)$. T satisfies the constraints of (I) if and only if

$$\begin{aligned} \ln(y_j)/-\alpha & \geq D_{1j} & \text{for } j \in F_1 \\ [\ln(y_j)/-\alpha] - [\ln(y_i)/-\alpha] & \geq D_{ij} & \text{for } i = 2, \dots, N \\ & & j \in F_i \\ + [\ln(y_N)/-\alpha] & \leq \delta. \end{aligned}$$

The constraints can be rearranged to say

$$\begin{aligned} \ln y_j & \leq -\alpha D_{1j} = \ln(K_{1j}^{-1}) & \text{for } j \in F_1 \\ \ln(y_j/y_i) = \ln y_j - \ln y_i & \leq -\alpha D_{ij} = \ln(K_{ij}^{-1}) & \text{for } i = 2, \dots, N \\ & & j \in F_i \\ \ln y_N & \geq -\alpha \delta = \ln(K_{N1}). \end{aligned}$$

Recall $K_{ij} = \exp(\alpha D_{ij})$ so $\ln K_{ij} = \alpha D_{ij}$; $\ln(K_{ij}^{-1}) = -\alpha D_{ij}$.

The natural logarithm is a strictly monotone transformation. If u_1 and u_2 are positive numbers, then $\ln u_1 < \ln u_2$ if and only if $u_1 < u_2$. This implies our constraints are equivalent to

$$\begin{aligned} y_j & \leq K_{1j}^{-1} & \text{for } j \in F_1 \\ y_j/y_i & \leq K_{ij}^{-1} & \text{for } i = 2, \dots, N \\ & & j \in F_i \\ y_N & \geq K_{N1}. \end{aligned}$$

We see that these are precisely the constraints of (II): multiply constraint j in the first set by K_{1j} , multiply constraint (i, j) in the second set by $y_i K_{ij}$, and multiply the last constraint by -1 . Q.E.D.

Let G be a matrix obtained from E as follows: if an element of E is 0 or -1 , the corresponding element of G is 0 or -1 ; if an element of E is $+1$ (in column (ij) and row j), then the corresponding element of G is $K_{ij} = \exp(\alpha D_{ij})$.

Let c be a vector defined on $\mathcal{A}$, $c_{1j} = 1$ for $j \in F_1$, $c_{N1} = -K_{N1}$, otherwise, $c_{ij} = 0$. Problems (II) become

(II)

 Maximize yq

 Subject to $yG \leq c$.

THEOREM (2): If $\delta_0 \leq \delta < +\infty$; T is an extreme point of $\{T \mid TE \geq D\}$ if and only if $y(T)$ is an extreme point of $\{y \mid yG \leq c\}$.

PROOF: Both E and G have rank $N-1$, thus y and T will be extreme points if and only if they satisfy at least $N-1$ linearly independent constraints with equality. Recall there is a one to one correspondence between constraints (columns of G and E) and arcs of $(V, \mathcal{A})$.

A tree in the project graph $(V, \mathcal{A})$ is a subgraph of $N-1$ arcs and no loops. Let $\mathcal{T}$ be the arcs in the tree and $\mathcal{T}' = \mathcal{A} \setminus \mathcal{T}$ the arcs deleted from $\mathcal{A}$ to form the tree. A tree is feasible if we can associate a schedule T with the tree so that

$$T_j - T_i = D_{ij} \quad \text{for } (i, j) \in \mathcal{T}$$

and

$$T_j - T_i \geq D_{ij} \quad \text{for } (i, j) \in \mathcal{T}',$$

where $T_1 = 0$. It is well known that the relations $T_1 = 0$ and $T_j - T_i = D_{ij}$ for $(i, j) \in \mathcal{T}$ uniquely determine the schedule T . Every extreme point of $\{T \mid TE \geq D\}$ is equivalent to a feasible tree.

Let T be determined by the feasible tree $(V, \mathcal{T})$. Thus, according to Theorem 1, $y(T)$ is in the set $\{y \mid yG \leq c\}$. We can show $y(T)$ is an extreme point by demonstrating that the $N-1$ constraints corresponding to arcs in $\mathcal{T}$ are linearly independent. Recall that $y(T)$ satisfies a constraint with equality if and only if T satisfies the corresponding constraint with equality. To show the constraints are linearly independent, we show that

$$\pi_j K_{ij} - \pi_i = 0 \quad (i, j) \in \mathcal{T}$$

$$\pi_1 = 0$$

has only one solution $(\pi_2, \dots, \pi_n) = 0$. Since $(V, \mathcal{T})$ is a tree, some arc is incident on node 1. For these arcs

$$\pi_j K_{1j} = \pi_1 = 0,$$

or

$$0 = \pi_1 K_{n1} = \pi_n.$$

Since $K_{ij} \neq 0$, $\pi_j = 0$ for all nodes in $\mathcal{T}$ incident on node 1. This argument can be continued by using the connectedness of $(V, \mathcal{T})$ to show $\pi_i = 0$ for all i . Therefore, $y(T)$ is an extreme point.

For the converse, suppose y is an extreme point. Since $\delta < \infty$, $K_{n1} > 0$, and $y_n \geq K_{n1} > 0$. Each

event is on a directed path from 1 to N , so $1 \geq y_j \geq y_n > 0$ for all j . Let $\mathcal{S}$ be the arcs corresponding to the $N-1$ tight, linearly independent constraints and let $\tilde{G}$ be the corresponding $(N-1) \times (N-1)$ submatrix of G . If $(V, \mathcal{S})$ is not a tree, the subgraph $(V, \mathcal{S})$ is disconnected. If $(V, \mathcal{S})$ contains an isolated node, then $\tilde{G}$ has a zero row contradicting linear independence.

Consider a connected component that *does not* contain node 1. For arcs in this component $y_j K_{ij} = y_i$, because $c_{ij} = 0$ unless i or j equals 1. Since $\tilde{G}$ is nonsingular, the only solution for these relation is $y_i = 0$ for nodes in the connected component. This is impossible since $y > 0$. Therefore, $(V, \mathcal{S})$ is a tree. Q.E.D.

As a consequence of Theorem 2, we can restrict our search for optimal schedules to feasible trees in the project graph. The algorithm described below does just that, while following the logic of the simplex method applied to problem (II).

Given a feasible tree $(V, \mathcal{T})$ we know T and $y(T)$. The optimality of $(V, \mathcal{T})$ is checked, using the standard complementary slackness results. Let $\tilde{G}$ be the submatrix of G corresponding to arcs in $\mathcal{T}$. If the $N-1$ vector x solving $\tilde{G}x = q$ is nonnegative, then the current solution is optimal. If some element of x is strictly negative, then the corresponding arc can be deleted from the tree and a better tree can be obtained.

Recall that $\mathcal{T}' = \mathcal{A}/\mathcal{T}$. We find x by setting $x_{ij} = 0$ for $(i, j) \in \mathcal{T}'$ and then solving

$$(1) \quad \sum_{j \in P_i} K_{ji} x_{ji} - \sum_{j \in F_i} x_{ij} = q_i$$

for $i = 2, 3, \dots, N$.

As an alternative to solving (1), we can, given T and thus $y(T)$, set $b_i = q_i y_i$ and solve

$$(2) \quad \sum_{j \in P_i} w_{ji} - \sum_{j \in F_i} w_{ij} = q_i y_i = b_i$$

for $i = 2, 3, \dots, N$.

For any feasible tree set $x_{ij} = w_{ij} = 0$ for $(i, j) \in \mathcal{T}'$. Let y be the extreme solution of II corresponding to $\mathcal{T}$. For $(i, j) \in \mathcal{T}$ set $x_{ij} y_i = w_{ij}$.

THEOREM (3): x_{ij} solves (1) if and only if w_{ij} solves (2).

PROOF: Substitute $x_{ij} y_i$ for w_{ij} in (2), and divide each equation by $y_i > 0$.

We obtain

$$\sum_{j \in P_i} (y_j/y_i) x_{ji} - \sum_{j \in F_i} x_{ij} = q_i$$

for $i = 2, \dots, N$.

If $x_{ji} > 0$, then $(j, i) \in \mathcal{T}$ and $y_j K_{ji} = y_i$, so $y_j/y_i = K_{ji}$. Therefore, x_{ij} solves (1). Note the steps are reversible if x_{ij} solves (1) then $K_{ji} x_{ji} = (y_j/y_i) x_{ji} = w_{ji}/y_i$. Q.E.D.

Since $w_{ij} \geq 0$ if $x_{ij} \geq 0$, optimality can be checked by solving (2). This saves work since (2) is a triangular system with only 0, and ± 1 for coefficients. If we find some $w_{lk} < 0$ for $(l, k) \in \mathcal{T}$ then the current tree is not optimal, removing arc (l, k) from $\mathcal{T}$ will disconnect the graph $(V, \mathcal{T})$. Suppose

$(U, \mathcal{S})$ and $(W, \mathcal{R})$ are the disconnected subgraphs and node 1 is in $(U, \mathcal{S})$. Two cases can be considered.

CASE 1: Node l is in U node $K \in W$. In the current schedule, we have $T_k - T_l = D_{lk}$. If the new T^* is going to be feasible, we must have $T_k^* - T_l^* \geq D_{lk}$. One arc from $\mathcal{T}'$ will be added to $(U, \mathcal{S})$ and $(W, \mathcal{R})$ to form the new tree. Obviously, this new arc will connect the disjoint subgraphs. All the arcs in $\mathcal{S}$ will be in the new tree. From this we can conclude that $T_i^* = T_i$ for all $i \in U$. This is true since all T_i for $i \in U$ are determined by $T_j - T_i = D_{ij}$ for $(i, j) \in \mathcal{S}$ and $T_1 = 0$.

In particular, T_l will not change; $T_l^* = T_l$. This means $T_k^* \geq T_k$ if the new tree is going to be feasible. If T_k increases, then to preserve the equation, $T_j - T_i = D_{ij}$ for $(i, j) \in \mathcal{R}$, we must have each T_i increasing (for $i \in W$) by the same amount as T_k . Thus the question is how much can we increase the T_i for $i \in W$ and still remain feasible. The only arcs we have to worry about are those running from W to U . The limit of the increase is given by

$$\theta = \text{Min } [T_j - T_i - D_{ij} | i \in W, j \in U, (i, j) \in \mathcal{T}'].$$

Suppose arc (m, n) achieves the minimum, then $\theta = T_m - T_n - D_{mn} \geq 0$. Arc (m, n) is added to $\mathcal{S}$ and $\mathcal{R}$ to form a new tree.

The new values of T and y are readily calculated.

CASE 2: $l \in W, k \in U$. The same reasoning applies. T_k must remain constant. All the T_i for $i \in W$ must decrease by a constant amount. Only arcs $(i, j) \in \mathcal{T}$ running from $(U, \mathcal{S})$ to $(W, \mathcal{R})$ need to be considered. The decrease is given by

$$\theta = \text{Max } [T_i + D_{ij} - T_j | i \in U, j \in W, (i, j) \in \mathcal{T}'].$$

If arc (m, n) achieves the maximum, then the new tree is formed by adding (m, n) to $\mathcal{S}$ and $\mathcal{R}$. Note that $\theta = T_m + D_{mn} - T_n$ is nonpositive. In either case for $i \in W$, the new epoch is $T_i + \theta$. Therefore, the new b vector for $i \in W$ is given by

$$q_i \exp [-\alpha (T_i + \theta)] = q_i \exp [-\alpha T_i] \exp [-\alpha \theta] = b_i \Delta,$$

where

$$\Delta = \exp [-\alpha \theta].$$

The increase in present value is $(\Delta - 1) \sum_{i \in W} b_i$.

PROPOSITION: The increase in present value is given by

(i) $(\Delta - 1) w_{lk}$ in case 1, and

(ii) $-(\Delta - 1) w_{lk}$ in case 2.

PROOF: We must show $\sum_{i \in W} b_i$ equals w_{lk} in case 1 and $-w_{lk}$ in case 2. To determine this sum, the relation in (2) for $i \in W$. If an arc in $\mathcal{T}$ runs from U to U , it is not counted. If it runs from W to W , it is counted once in a positive sense and once in a negative sense. Arc (l, k) is the only arc in $\mathcal{T}$ connecting U and W . If $l \in U$ and $k \in W$ (case 1), then at node $k \in W$, we obtain a contribution w_{lk} . Therefore,

$$\sum_{i \in W} b_i = \sum_{i \in W} \left[\sum_{j \in P_i} w_{ji} - \sum_{j \in F_i} w_{ij} \right] = \begin{cases} w_{lk} & \text{in case 1} \\ -w_{lk} & \text{in case 2} \end{cases}.$$

Note, if $\theta=0$, that a positive increase in present value is made at each step. In case (1) $\theta > 0$, thus $\Delta < 1$, so $(\Delta - 1) < 0$, $w_{lk} < 0$ implies $(\Delta - 1) w_{lk} > 0$. In case 2 $\theta < 0$, $\Delta > 1$, so $-(\Delta - 1) < 0$, $w_{lk} < 0$ implies $-(\Delta - 1)w_{lk} > 0$. Q.E.D.

Note the economic interpretation. In case 1, we have isolated a set of nodes (W , not containing node 1) with a net loss of present value. The algorithm directs us to delay that loss as much as possible. In case 2, we find a set of nodes (W) with a net gain in present value. The algorithm directs us to advance the scheduling of events in W .

Let $PV(D)$ be the optimal value of problem (I) as a function of the duration vector D . As Russell has indicated, the w_{ij} can be interpreted as

$$-\frac{\partial PV(D)}{\partial D_{ij}} = w_{ij}/\alpha.$$

Thus w_{ij}/α is the marginal present value return of shortening the duration of activity (i, j) . If $D_{ij}^* = D_{ij} - \Delta$, then $PV(D^*) \cong PV(D) + (\Delta, \alpha)w_{ij}$. Since $x_{ij}y_i = w_{ij}$, this indicates

$$x_{ij}/\alpha = -\frac{\partial PV(D)}{\partial D_{ij}} \cdot \exp(+\alpha T_i).$$

The x_{ij}/α measures the marginal return at time T_i that is obtained by shortening the duration of activity (i, j) .

THE FIXED DEADLINE ALGORITHM

PHASE 1: Find the early tree ([3], pg 27) $(V, \mathcal{T})$ and the early start times T . Set $T_N = \delta_0$; if $\delta_0 > \delta$ no schedule can meet the deadline. Otherwise calculate $b_i = q_i \exp(-\alpha T_i)$ for $i = 1, 2, \dots, N$; set $PV = \sum_{i=2}^N b_i$.

PHASE 2:

STEP 1. *Finding an arc to delete from the feasible tree.* Set $w_{ij} = 0$ for $(i, j) \in \mathcal{T}'$, then solve

$$\sum_{j \in P_i} w_{ij} - \sum_{j \in F_i} w_{ij} = b_i \quad \text{for } i = 2, 3, \dots, N.$$

If all $w_{ij} \geq 0$, the current solution is optimal. If some $w_{lk} < 0$, go to step 2.

STEP 2. *Finding a new arc.* Deleting arc (l, k) from $(V, \mathcal{T})$ forms subgraphs $(U, \mathcal{S})$ and $(W, \mathcal{R})$ where node 1 is in U . If $l \in U$ go to step 3; if $l \in W$ go to step 4.

STEP 3. *Delay the events in W .* Compute

$$\theta = \text{Min}[T_j - T_i - D_{ij} \mid i \in W, j \in U, (i, j) \in \mathcal{T}'].$$

Suppose arc (m, n) achieves the minimum. Go to step 5.

STEP 4. *Advance the events in W .* Compute

$$\theta = \text{Max}[T_i + D_{ij} - T_j \mid i \in U, j \in W, (i, j) \in \mathcal{T}'].$$

Suppose arc (m, n) achieves the maximum. Go to step 5.

STEP 5. *Change of schedule.* Let $\Delta = \exp(-\alpha\theta)$. For $i \in W$,

$$T_i \leftarrow T_i + \theta;$$

$$b_i \leftarrow b_i \Delta;$$

Delete (l, k) from $\mathcal{T}$ and add (m, n) . Go to step 1.

We can note that:

(i). If $\delta = \delta_0$, the arc $(N, 1)$ is likely to be in the optimal tree. In this case, Phase 1 could be modified as follows: find the late tree, delete the arc on the critical path joining node 1, add arc $(N, 1)$, set T equal to the late start times.

(ii). Sensitivity of the optimal schedule to changes in α can be determined from (2). Keep T fixed set $w_{ij} = 0$ for $(i, j) \in \mathcal{T}'$, then determine the range of α for which (2) has a nonnegative solution. For example if α is increased until some w_{ij} becomes 0, then the algorithm described above can be used to find the new optimal tree. Unfortunately this sensitivity procedure involves some difficult calculations.

THE PARAMETRIC ALGORITHM

It may be desirable to solve problem (I) parametrically in the deadline constraint to measure the exact trade-off between present value and completion time. This section presents such an algorithm.

We assume the procedure described in the last section has been used to find an optimal schedule for $\delta = \delta_0$. Then, δ is increased to $+\infty$. One of two things will happen; either (i) the optimal solution will become insensitive to further increases in the deadline, or (ii) we will discover that the deadline constraint will be binding for any limit. This implies it is not optimal to complete the project if no deadline is imposed.

We find that the optimal value $PV(\delta)$ is a concave increasing function of δ made up of exponential segments. If a tree is optimal in the region $\delta_h \leq \delta \leq \delta_{h+1}$ and W_{N1} is in the solution related to this tree, then

$$PV(\delta) = [(1 - \exp(-\alpha[\delta - \delta_h]))] \alpha \cdot w_{N1} + PV(\delta_h)$$

for $\delta_h \leq \delta \leq \delta_{h+1}$.

The parametric procedure follows the logic of the simplex method while exploiting the special structure of our problem. We assume Phase 1 and Phase 2 have been completed.

PHASE 3:

STEP 0. *Start.* From Phase 1 and 2 we know δ_0 and the optimal tree $(V, \mathcal{T})$ when $\delta = \delta_0$. Let PV , T , and w be the solution obtained from Phase 2. Set $h = 0$, $PV_0 = PV$.

STEP 1: *Optimality Check, Find arc to join tree.* If $w_{N1} = 0$, the current schedule T is optimal for all $\delta \geq \delta_h$. Otherwise, delete arc $(N, 1)$ from $(V, \mathcal{T})$ and let $(U, \mathcal{S})$ and $(W, \mathcal{R})$ be the disconnected subgraphs; $1 \in U$, $N \in W$. If no arc, aside from $(N, 1)$, goes from W to U , then the current tree is optimal

for all $\delta \geq \delta_h$. Let $\theta = \delta - \delta_h$. The optimal schedule is

$$T_i(\theta) = \begin{cases} T_i, & \text{if } i \in U \\ T_i + \theta, & \text{if } i \in W \end{cases}.$$

If some arc goes from W to U , compute

$$\theta = \text{Max}[T_j - T_i - D_{ij} \mid i \in W, j \in U, (i, j) \in \mathcal{T}'].$$

Suppose $\theta = T_n - T_m - D_{mn} \geq 0$.

STEP 2. *New solution.* Set

$$\Delta \leftarrow \exp(-\alpha\theta)$$

$$\delta_{h+1} \leftarrow \delta_h + \theta$$

$$PV_{h+1} \leftarrow PV_h + (1 - \Delta)w_{N1}$$

$$h \leftarrow h + 1.$$

For $i \in W$, set

$$T_i \leftarrow T_i + \theta$$

$$W_{ij} \leftarrow w_{ij}\Delta.$$

STEP 3. *Finding an arc to drop.* Adding (m, n) to $(V, \mathcal{T})$ forms a circuit $\mathcal{C}$. Note that (m, n) and $(N, 1)$ are in $\mathcal{C}$ and that the arc oriented in opposite fashions. Let $\mathcal{C}'$ be all arcs in $\mathcal{C}$ oriented as (m, n) and $\mathcal{C}^2$ the arcs oriented as $(N, 1)$. Find

$$\theta = \text{Min} \{w_{ij} \mid (i, j) \in \mathcal{C}^2\}.$$

Suppose $\theta = w_{lk} \geq 0$.

For $(i, j) \in \mathcal{C}^1$

$$w_{ij} \leftarrow w_{ij} + \theta;$$

For $(i, j) \in \mathcal{C}^2$

$$w_{ij} \leftarrow w_{ij} - \epsilon,$$

go to step 1.

APPENDIX

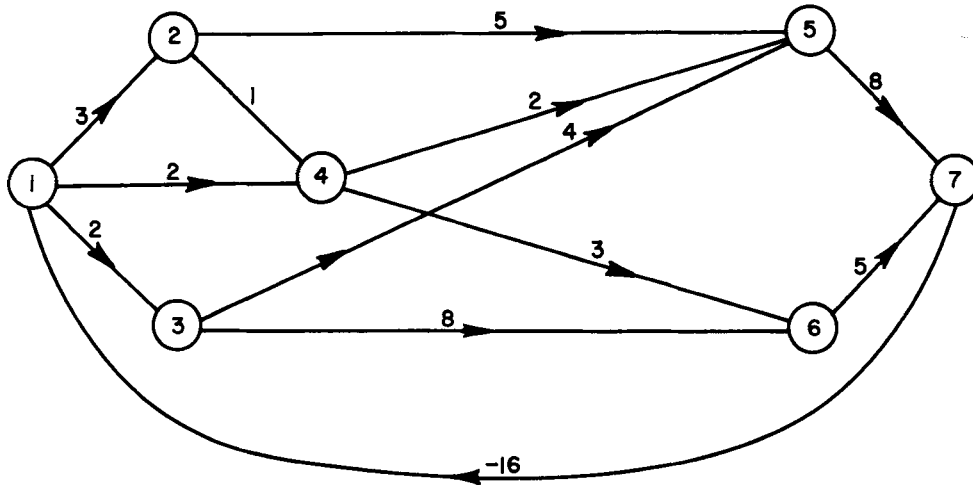
AN EXAMPLE

This section gives an example of payment scheduling problems. There are 7 events, 11 activities, and a deadline on the project. Thus the project graph has 7 nodes and 12 arcs.

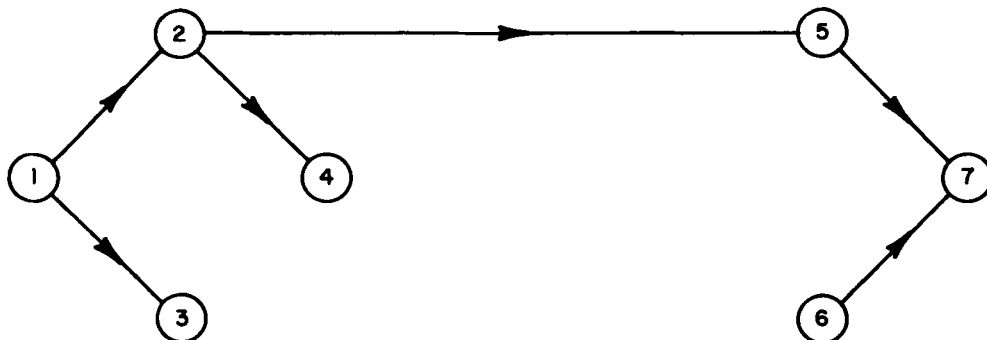
Data for the problem are summarized in the table and figure following. We are using the same problem as Russell [2, p. 370]. The discount factor is $\alpha = 0.01$; time is measured in months. Russell assumes $q_1 = -100$. We take $q_1 = 0$. Thus the value of our solution will be 100 greater than Russell's.

Event	P_i	F_i	q_i	Early starts
1	7	2, 3, 4	0	0
2	1	4, 5	-200	3
3	1	5, 6	-200	2
4	1, 2	5, 6	100	2
5	2, 3, 4	7	400	8
6	3, 4	7	-200	10
7	5, 6	1	300	16

The network is drawn below; the numbers on the arcs are the D_{ij} . Note the deadline is 16.

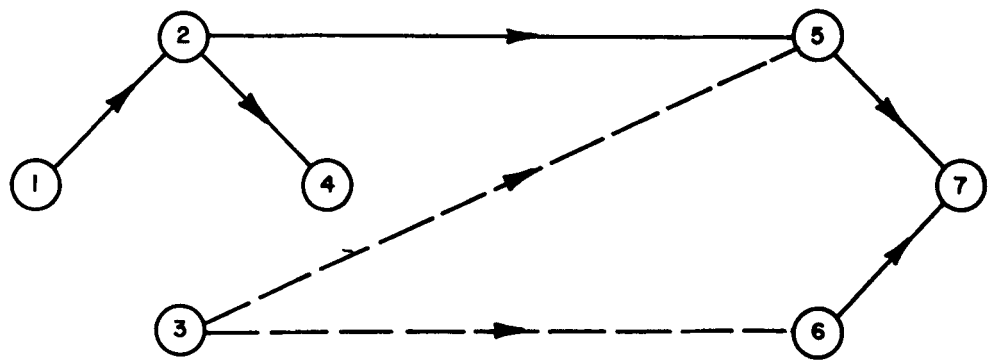


We show one iteration of the fixed deadline algorithm. A feasible tree and the associated values of T and b are shown below. The present value of this solution is 151.69.



	2	3	4	5	6	7
<i>T</i>	3	2	4	8	11	16
<i>b</i>	-194.09	196.04	96.08	369.24	-179.16	255.66

On solving (2) we find that $w_{13} = -196.04$. Arc (1, 3) is detached from the tree. The possible incoming arcs are dotted.



According to Step 3,

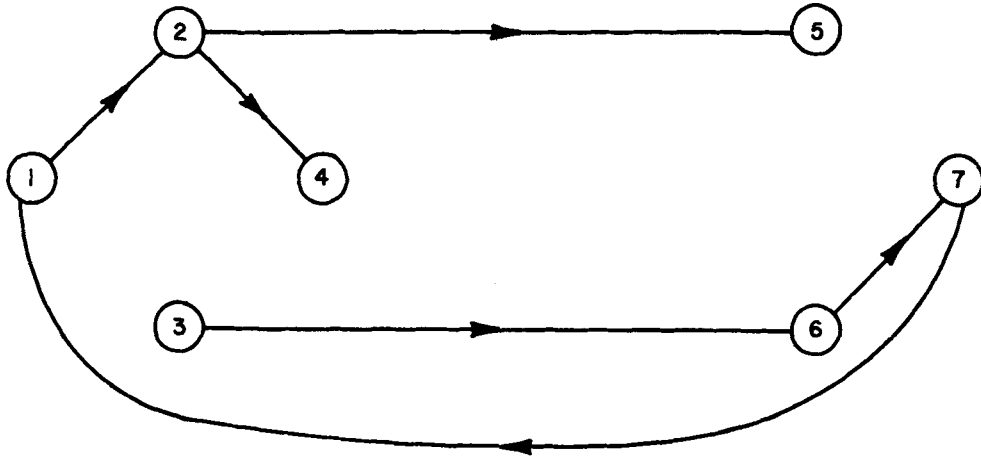
$$\theta = \text{Min} \begin{bmatrix} T_5 - T_3 - D_{35} = 2 \\ T_6 - T_3 - D_{63} = 1 \end{bmatrix} = 1.$$

Arc (3, 6) joins *th* tree. The new values of *b* and *T* are tabulated below. The present value of the new schedule is 153.64.

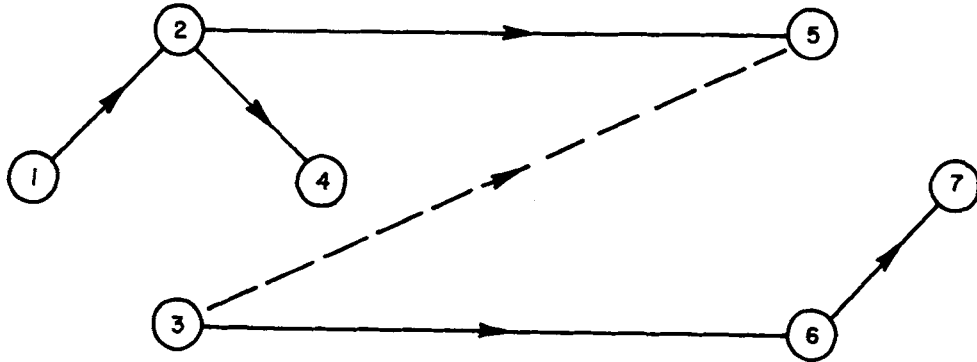
	2	3	4	5	6	7
<i>T</i>	3	3	4	8	11	16
<i>b</i>	-194.09	-194.09	96.08	369.24	-179.16	255.66

These values of *T* and *b* are optimal although we must go through one degenerate step to get the optimal values of *w* and the final tree.

	(1,2)	(2,4)	(2,5)	(3,6)	(6,7)	(7,1)
<i>w</i>	271.23	96.08	369.24	194.09	373.25	117.59



Now, we apply the parametric algorithm to our problem: $\delta_0 = 16$ and $PV_0 = 153.64$. Since $w_{71} \approx 117.59 > 0$, we look at the subgraphs formed by deleting $(7,1)$.



The only arc, except 71 , from $(W, \mathcal{R})$ to $(U, \mathcal{S})$ is $(3,5)$. Thus

$$\theta = T_5 - T_3 - D_{35} = 1.$$

In Step 2, we find the new values $\Delta = 0.99$, $\delta_1 = 17$, and $PV_1 = 154.80$. The new values of T and w are

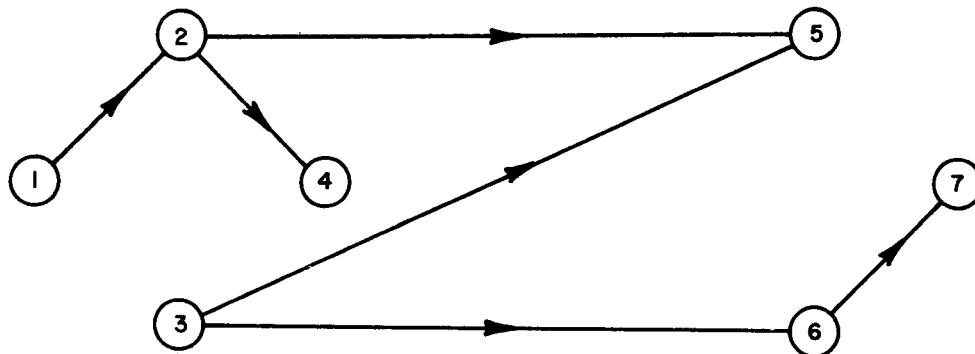
	2	3	4	5	6	7
T	3	4	4	8	12	17

	(1,2)	(2,4)	(2,5)	(3,6)	(6,7)	(7,1)
w	271.23	96.08	369.24	192.16	369.54	116.43

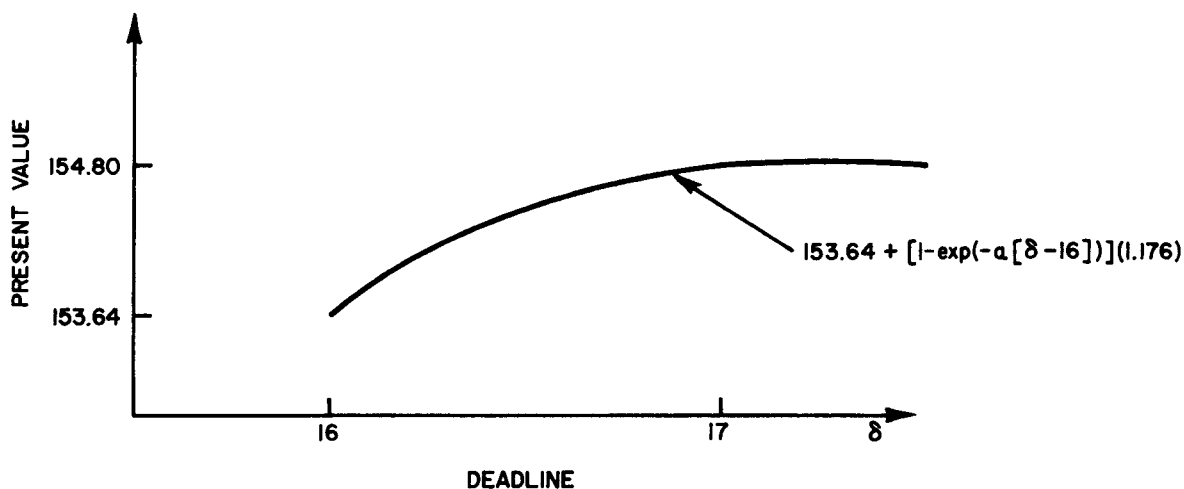
Notice that $PV_1 = w_{12} - w_{71}$. Adding arc $(3,5)$ creates a circuit with $C^1 = \{(3,5)\}$; $C^2 = \{(7,1), (1,2), (2,5), (3,6), (6,7)\}$. Thus $\theta = w_{71} = 116.43$ and the new values of w are

	(1,2)	(2,4)	(2,5)	(3,6)	(6,7)	(3,5)
w	154.79	96.08	252.81	75.73	253.11	116.43

The tree is



Since $w_{71} = 0$, this solution is optimal for all values of $\delta \geq 17$. The plot of present value against deadline is



ACKNOWLEDGMENT

I express my appreciation to the referee for his thorough report.

REFERENCES

- [1] Dantzig, G. B., *Linear Programming and Extensions* (Princeton University Press, New Jersey, 1963).
- [2] Russell, A. H., "Cash Flows in Networks," *Management Science*, 16, 357-73 (Jan. 1970).
- [3] Weist, J. D. and F. K. Levy, *A Management Guide to PERT/CPM* (Prentice-Hall, 1969).